



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

Python Programming for AI Applications

Eng. Fatma Elraey*Day 5 — Introduction to NumPy (Working with Arrays)*

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications (6-Day Course)
Session	Day 5 of 6
Focus	A gentle, hands-on introduction to NumPy arrays — the building block every AI and data library (pandas, scikit-learn, TensorFlow, PyTorch) is built on top of.

Quick Recap — Day 4

On Day 4 you learned to bundle data and behavior together using classes — constructors, inheritance, polymorphism, encapsulation. Today we take a break from building your own objects and start using one of the most important objects in all of AI programming: the NumPy array. Don't worry if you've never heard of NumPy before — we will start from zero and keep things simple.

Day 5 Outline

1. Why NumPy? Lists vs. Arrays
2. Creating Your First Arrays
3. Shapes, Indexing & Slicing
4. Array Math & Broadcasting
5. Summarizing Data with Aggregations
6. Useful Everyday Tips



01

Why NumPy? Lists vs. Arrays

So far, whenever you needed to store a bunch of numbers, you used a Python list. Lists are great for general-purpose storage, but they were never designed for heavy number-crunching. NumPy (short for Numerical Python) gives you a new kind of list, called an array, that is built specifically for fast math on lots of numbers at once.

The first thing to install and import — like any library, NumPy needs to be imported before you can use it. Almost everyone imports it with the short nickname np:

```
import numpy as np
```

Let's see the difference in action. Say you have five quiz scores and want to double every score:

```
scores = [4, 8, 15, 16, 23]
print(scores * 2)
# [4, 8, 15, 16, 23, 4, 8, 15, 16, 23]
# A plain list just repeats itself – not what we wanted!

scores = np.array([4, 8, 15, 16, 23])
print(scores * 2)
# [ 8 16 30 32 46]
# A NumPy array multiplies every number, exactly what we wanted.
```

Python list



*scores * 2 → repeats the list, doesn't multiply*

[4, 8, 15, 16, 23, 4, 8, 15, 16, 23]

NumPy array



*scores * 2 → multiplies every number at once*

[8, 16, 30, 32, 46]

Same data, very different behavior — NumPy math applies to every element automatically, and runs far faster on large data.

Figure 5.1 — Same data, very different behavior.

`np.array()` is how you turn a normal Python list into a NumPy array. From that point on, math operators like `+`, `-`, `*`, and `/` apply to every number inside the array automatically — no loops needed.



AI CONNECTION *Every image, every table of sensor readings, every batch of numbers fed into a machine learning model starts life as a NumPy array (or something built on top of one, like a pandas DataFrame or a PyTorch tensor). Learning arrays now means every AI library afterward will feel familiar.*

✓ QUICK CHECK — Why NumPy? Lists vs. Arrays

Q1. What does `scores * 2` do to a plain Python list vs. a NumPy array?

Answer: *On a list it repeats the list twice; on a NumPy array it multiplies every number by 2.*

Q2. What function turns a Python list into a NumPy array?

Answer: `np.array(your_list)`

02

Creating Your First Arrays

Besides converting a list, NumPy has several handy shortcuts for creating arrays without typing out every number by hand:

```
np.zeros(5)          # array([0., 0., 0., 0., 0.])
np.ones(3)           # array([1., 1., 1.])
np.arange(0, 10, 2)  # array([0, 2, 4, 6, 8])
np.linspace(0, 1, 5) # array([0., 0.25, 0.5, 0.75, 1.])
```

`np.zeros(n)` and `np.ones(n)` create an array of a given length filled with 0s or 1s — handy starting points. `np.arange(start, stop, step)` counts up like Python's `range()`, but returns an array. `np.linspace(start, stop, count)` gives you a fixed number of evenly spaced values between two numbers, which is often more useful than guessing a step size.

Every array has a few properties worth knowing right away — its shape, its number of dimensions, and its data type:

```
scores = np.array([90, 85, 78, 92, 88])
print(scores.shape) # (5,) -- 5 numbers, in one dimension
print(scores.ndim)  # 1    -- one dimension (a simple row)
print(scores.dtype) # int64 -- the numbers are whole integers
```

1D array — shape (5,)

90	85	78	92	88
----	----	----	----	----

one row of exam scores

2D array — shape (3, 4)

90	85	78	92
70	88	95	60
82	77	91	84

3 students × 4 quizzes each

Figure 5.2 — A 1D array is a single row; a 2D array is a grid of rows and columns.

Arrays can also have more than one dimension. A 2D array looks like a grid — think of it as a table of numbers, like three students' scores on four quizzes each:

```
quiz_grid = np.array([
    [90, 85, 78, 92],
    [70, 88, 95, 60],
    [82, 77, 91, 84],
])
print(quiz_grid.shape) # (3, 4) -- 3 rows, 4 columns
```

AI CONNECTION *A grayscale image is nothing more than a 2D array of brightness values, and a color image is a 3D array (height, width, and 3 color channels). Every dataset you'll load for a machine learning model is shaped this way — rows for examples, columns for features.*

✓ QUICK CHECK — Creating Your First Arrays

Q1. What does `np.arange(0, 10, 2)` produce?

Answer: `array([0, 2, 4, 6, 8])` — it counts from 0 up to (but not including) 10, in steps of 2.

Q2. What does `.shape` tell you about an array?

Answer: The size of each dimension — for example (3, 4) means 3 rows and 4 columns.

03

Shapes, Indexing & Slicing

Reading values out of a NumPy array works a lot like a Python list — square brackets, and a starting index of 0.

```
arr = np.array([10, 20, 30, 40, 50, 60])
print(arr[0])    # 10 -- the first item
print(arr[-1])   # 60 -- the last item
print(arr[1:4])  # [20 30 40] -- a slice
```



Slicing works just like Python lists — start:stop keeps start, stops one before stop.

Figure 5.3 — Indexing and slicing behave exactly like Python lists.

For a 2D array, you give both a row and a column position, separated by a comma, instead of using two separate square brackets:

```
quiz_grid = np.array([
    [90, 85, 78, 92],
    [70, 88, 95, 60],
    [82, 77, 91, 84],
])
print(quiz_grid[0, 2]) # 78 -- row 0, column 2
print(quiz_grid[1])    # [70 88 95 60] -- the whole 2nd row
print(quiz_grid[:, 0])  # [90 70 82] -- the whole 1st column
```

A lone colon (:) means "give me everything along this dimension." So [:, 0] reads as: every row, but only column 0.

One more very useful trick: boolean indexing. You can filter an array using a plain comparison, and NumPy will keep only the values where the comparison is true:

```
scores = np.array([55, 82, 91, 40, 76])
passed = scores[scores >= 60]
print(passed) # [82 91 76]
```



AI CONNECTION *scores >= 60 quietly builds an array of True/False values, one for each score — this is exactly the mechanism behind filtering a dataset for "rows where age > 30" or "pixels brighter than a threshold." You'll see this pattern everywhere once you start using pandas.*

✓ QUICK CHECK — Shapes, Indexing & Slicing

Q1. For a 2D array grid, what does grid[1] return?

Answer: *The entire second row (index 1) as a 1D array.*

Q2. What does grid[:, 0] mean?

Answer: *Every row, but only column 0 — i.e. the whole first column.*

Q3. What does scores[scores >= 60] do?

Answer: *Keeps only the values in scores that are greater than or equal to 60 (boolean/filter indexing).*

04

Array Math & Broadcasting

You already saw that `scores * 2` multiplies every number. This works for all the basic math operators, and also between two arrays of the same size, element by element:

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b) # [11 22 33]
print(a * b) # [10 40 90]
```

But what happens when the two things being combined are different sizes, like an array and a single number? NumPy uses a rule called broadcasting: the smaller value is automatically "stretched" to match, without actually copying any data.



The scalar "stretches" to match every element — that's broadcasting.

Figure 5.4 — The scalar 5 stretches to line up with every element.

```
scores = np.array([70, 65, 80, 55])
curved = scores + 5 # add 5 bonus points to every score
print(curved) # [75 70 85 60]
```

Broadcasting also works between a 2D array and a 1D array, as long as the shapes line up sensibly — for example, adding a single row of bonus points to every row of a grid:

```
quiz_grid = np.array([
    [90, 85, 78, 92],
    [70, 88, 95, 60],
])
bonus = np.array([2, 2, 0, 5])
print(quiz_grid + bonus)
# [[92 87 78 97]
#   [72 90 95 65]]
```




AI CONNECTION *Broadcasting is why you can normalize an entire dataset in one line — subtracting the mean and dividing by the standard deviation across thousands of rows happens without a single explicit loop, which is a big part of why NumPy code runs so much faster than plain Python.*

✓ QUICK CHECK — Array Math & Broadcasting

Q1. What happens when you do array + array of the same shape?

Answer: *The operation happens element by element — each pair of matching positions is added together.*

Q2. What is broadcasting?

Answer: *NumPy's rule for automatically 'stretching' a smaller array (like a single number) to match a bigger array's shape, so the operation can apply to every element.*

05

Summarizing Data with Aggregations

An aggregation takes many numbers and boils them down to one summary number. NumPy arrays come with these built right in as methods:

```
scores = np.array([90, 85, 78, 92, 88])
print(scores.sum()) # 433
print(scores.mean()) # 86.6
print(scores.max()) # 92
print(scores.min()) # 78
print(scores.std()) # 4.89 (roughly)
```

On a 2D array, these same functions can work on the whole grid, or along just one direction (called an axis). axis=0 walks down the columns; axis=1 walks across the rows.

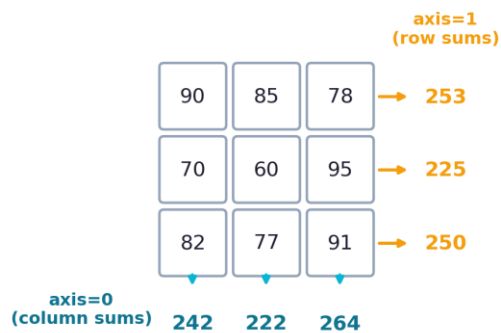


Figure 5.5 — axis=1 adds across each row; axis=0 adds down each column.

```
quiz_grid = np.array([
    [90, 85, 78],
    [70, 60, 95],
    [82, 77, 91],
])
print(quiz_grid.sum()) # 728 -- every number added
print(quiz_grid.sum(axis=1)) # [253 225 250] -- total per student
print(quiz_grid.mean(axis=0)) # [80.67 74. 88.] -- average per quiz
```

A good way to remember which axis is which: axis=1 moves along the row (left to right), so it collapses each row down to one number — one total per student. axis=0 moves down the column, collapsing each column down to one number — one average per quiz.

AI CONNECTION When you evaluate a trained model, computing "accuracy per class" or "average loss per batch" almost always comes down to calling `.mean()` or `.sum()` along a specific axis of a results array.



✓ QUICK CHECK — Summarizing Data with Aggregations

Q1. What does `scores.mean()` return?

Answer: *The average of every value in the array.*

Q2. For a 2D grid, what does `axis=1` collapse — rows or columns?

Answer: *It collapses each row down to a single number, moving across the row (left to right).*

06

Useful Everyday Tips

`reshape()` lets you rearrange the same numbers into a different shape, as long as the total count of numbers stays the same:

```
nums = np.arange(1, 13)
print(nums) # [ 1  2  3  4  5  6  7  8  9 10 11 12]

grid = nums.reshape(3, 4)
print(grid)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
```

`np.random` lets you generate quick fake data for testing — very handy while learning or prototyping:

```
np.random.seed(0) # makes the "random" numbers repeatable
dice = np.random.randint(1, 7, size=5)
print(dice) # [5 6 1 4 4] (five random rolls, 1 to 6)
```

Two arrays can be compared and sorted just as easily as one:

```
scores = np.array([90, 85, 78, 92, 88])
print(np.sort(scores)) # [78 85 88 90 92]
print(scores.argmax()) # 3 -- the index of the highest score
```

AI CONNECTION *`np.random` is exactly how you'll generate the small toy datasets used to test a model's code before it ever touches real data, and `reshape()` is how a flat list of pixel values gets turned back into a proper height × width image before it's shown or processed.*

✓ QUICK CHECK — Useful Everyday Tips

Q1. What does `reshape(3, 4)` require about the original array?

Answer: It must have exactly $3 \times 4 = 12$ numbers total — `reshape` rearranges the same data, it doesn't add or remove any.

Q2. Why call `np.random.seed(0)` before generating random numbers?

Answer: It makes the 'random' results repeatable — running the code again produces the exact same numbers, which is useful for testing.



Practical Time

Work through these on your own before checking the marked answer.

Practice Question 1

What will this code print?

```
a = [1, 2, 3]
print(a * 2)
```

- A. [2, 4, 6]
- B. [1, 2, 3, 1, 2, 3] ✓
- C. TypeError
- D. [1, 2, 3, 2, 4, 6]

*Why: a is a plain Python list, not a NumPy array, so * 2 repeats the list rather than multiplying each number.*

Practice Question 2

What will this code print?

```
import numpy as np
arr = np.array([2, 4, 6, 8])
print(arr[1:3])
```

- A. [4 6] ✓
- B. [2 4 6]
- C. [4 6 8]
- D. [2 4]

Why: Slicing [1:3] starts at index 1 and stops before index 3, giving the values at index 1 and 2: 4 and 6.

Practice Question 3

What is the shape of this array?

```
grid = np.array([
    [1, 2, 3],
    [4, 5, 6],
])
```

- A. (3, 2)
- B. (2, 3) ✓**
- C. (6,)
- D. (2, 2)

Why: shape reports (rows, columns). There are 2 rows and 3 columns, so the shape is (2, 3).

Practice Question 4

What will this code print?

```
scores = np.array([40, 55, 70, 90])
print(scores[scores >= 60])
```

- A. [True True]
- B. [70 90] ✓**
- C. [40 55]
- D. IndexError

Why: scores >= 60 builds a True/False array, and using it inside [] keeps only the values where it's True — 70 and 90.

Practice Question 5

What will this code print?

```
prices = np.array([10, 20, 30])
print(prices + 5)
```

- A. TypeError, arrays can't add numbers
- B. [15, 20, 30]
- C. [15 25 35] ✓**
- D. [10, 20, 30, 5]

Why: Broadcasting stretches the single number 5 to match every element, so it's added to each price individually.

Practice Question 6

For `grid.sum(axis=1)` on a 2D array, what gets collapsed?

- A. Each column, giving one total per column
- B. Each row, giving one total per row ✓**
- C. The whole array into a single number
- D. Nothing — `axis=1` is invalid for `sum()`

Why: `axis=1` moves across each row, so summing along it collapses each row down to one total — one value per row.